

# PROJE GÜVENLİK VE TEST RAPORU

Online Eğitim Platformu

**Proje Adı:** Online Eğitim Platformu

**Rapor Versiyonu:** v2.0 - Detaylı Analiz

**Hazırlayan:** Zübeyir Aslan

**Tarih:** 06 / 05 / 2026

**Kapsam:** Backend (Node.js/Express) + Frontend (React/Vite)

**Kullanılan Araçlar:** npm audit, Snyk, ESLint, Retire.js

Bu rapor, Online Eğitim Platformu projesi için yapılan statik kod analizi, bağımlılık taraması ve güvenlik denetimleri sonuçlarını içermektedir.

# İÇİNDEKİLER

---

|                                             |    |
|---------------------------------------------|----|
| 1. Yönetici Özeti                           | 3  |
| 2. Kullanılan Araçlar ve Yöntemler          | 4  |
| 3. Bağımlılık Tarama Sonuçları (npm audit)  | 5  |
| 4. Snyk ile Detaylı Zafiyet Analizi         | 7  |
| 5. Statik Kod Analizi (ESLint)              | 9  |
| 6. Frontend Güvenlik Taraması (Retire.js)   | 10 |
| 7. Tespit Edilen Kritik Güvenlik Açıkları   | 11 |
| 8. Uygulanan Çözümler ve Düzeltmeler        | 13 |
| 9. Yazılım Mühendisliği ve Test Prensipleri | 15 |
| 10. Sonuç ve Öneriler                       | 17 |

# 1. Yönetici Özeti

Bu rapor, Online Eğitim Platformu projesinde gerçekleştirilen kapsamlı statik kod analizi ve güvenlik denetimi çalışmalarının sonuçlarını sunmaktadır. Çalışma kapsamında hem backend (Node.js/Express) hem de frontend (React/Vite) bileşenleri detaylı şekilde incelenmiş; endüstri standartı araçlar olan npm audit, Snyk, ESLint ve Retire.js kullanılarak çoklu katmanlı bir güvenlik taraması uygulanmıştır.

## Genel Bulgu Özeti

| Tarama Türü          | Tespit Edilen Sorun                    | Durum             |
|----------------------|----------------------------------------|-------------------|
| Backend - npm audit  | 12 zafiyet (2 yüksek, 5 orta, 5 düşük) | Kısmen Düzeltildi |
| Backend - Snyk       | 20 zafiyet, 23 saldırı vektörü         | Analiz Edildi     |
| Frontend - npm audit | 0 zafiyet (Vite güncellendi)           | Temiz             |
| Frontend - Retire.js | 0 zafiyetli kütüphane                  | Temiz             |
| Statik Kod (ESLint)  | Kod taraması başarılı                  | Aktif             |

## Önemli Çıkarımlar

- Frontend tarafı tamamen güvenli hale getirildi: Vite güncellemesi sonrası 0 zafiyet tespit edildi.
- Backend tarafında kritik bir keşif yapıldı: Tüm zafiyetlerin temel kaynağı **express-status-monitor** paketinin eski axios@0.26.0 sürümüne bağımlı olmasıdır.
- Snyk taraması ile prototype pollution, SSRF, CSRF, ReDoS gibi 20 farklı saldırı vektörü detaylı şekilde belgelendi.
- Manuel kod incelemesi ile şifre hashleme eksikliği, mass assignment ve kimlik doğrulama eksikliği gibi mimari seviyede sorunlar tespit edilmiştir.

## 2. Kullanılan Araçlar ve Yöntemler

Statik kod analizi, kodun çalıştırılmadan incelenerek güvenlik açıklarının tespit edilmesi sürecidir. Bu projede aşağıdaki dört ana araç birbirini tamamlayacak şekilde kullanılmıştır:

| Araç                            | Amaç                                              | Tarama Türü |
|---------------------------------|---------------------------------------------------|-------------|
| <b>npm audit</b>                | Bağımlılıklardaki bilinen zafiyetleri tespit eder | SCA*        |
| <b>Snyk</b>                     | Bağımlılık + kod düzeyi detaylı zafiyet analizi   | SCA + SAST  |
| <b>ESLint + Security Plugin</b> | JavaScript kod kalitesi ve güvenlik pattern'leri  | SAST**      |
| <b>Retire.js</b>                | Eski/zafiyetli JS kütüphanelerini tespit eder     | SCA         |

\* SCA: Software Composition Analysis (Bileşen analizi - üçüncü parti kütüphaneler)

\*\* SAST: Static Application Security Testing (Statik uygulama güvenlik testi - kendi kodumuz)

### Tarama Metodolojisi

Tarama süreci üç fazda gerçekleştirilmiştir:

- Faz 1 - Keşif: Backend ve frontend dizinlerinde mevcut bağımlılıklar listelendi ve npm audit ile ilk tarama yapıldı.
- Faz 2 - Derinlemesine Analiz: Snyk ile bağımlılık ağacı detaylı incelendi, zafiyetlerin türleri (CWE kategorileri) çıkarıldı.
- Faz 3 - Düzeltme ve Doğrulama: npm audit fix ile otomatik düzeltmeler uygulandı, breaking change gerektiren zafiyetler manuel olarak değerlendirildi.

## 3. Bağımlılık Tarama Sonuçları (npm audit)

### 3.1 İlk Tarama Sonucu

Backend dizininde çalıştırılan npm audit komutu sonucunda toplam 12 zafiyet tespit edilmiştir. Bu zafiyetlerin dağılımı aşağıdaki gibidir:

| Seviye          | Adet      | Durum                   |
|-----------------|-----------|-------------------------|
| Yüksek (High)   | 2         | Kritik müdahale gerekli |
| Orta (Moderate) | 5         | Acil müdahale önerilir  |
| Düşük (Low)     | 5         | Planlı düzeltme         |
| <b>Toplam</b>   | <b>12</b> | -                       |

### 3.2 Tespit Edilen Zafiyetli Paketler

| Paket            | Versiyon    | Seviye | Ana Sorun                                       |
|------------------|-------------|--------|-------------------------------------------------|
| axios            | <=0.31.0    | Yüksek | 15 farklı CVE (CSRF, SSRF, Prototype Pollution) |
| cookie           | <0.7.0      | Düşük  | Out of bounds karakter kabul (XSS)              |
| debug            | 4.0.0-4.3.0 | Düşük  | Regex DoS (ReDoS)                               |
| follow-redirects | <=1.15.11   | Orta   | Custom Auth Header sızıntısı                    |
| on-headers       | <1.1.0      | Düşük  | HTTP Response Header manipülasyonu              |
| parseuri         | <2.0.0      | Orta   | Regex DoS (ReDoS)                               |
| engine.io        | 1.8.0-6.6.1 | Düşük  | Bağımlı zafiyetler                              |
| socket.io        | 1.0.0-4.7.5 | Düşük  | Bağımlı zafiyetler                              |

### 3.3 Kritik Keşif: Bağımlılık Zinciri

Önemli bulgu: Tespit edilen zafiyetlerin neredeyse tamamı, projede kullanılan **express-status-monitor@1.3.4** paketinin eski alt bağımlılıklarına sahip olmasından kaynaklanmaktadır. Bu durum, dolaylı (transitive) bağımlılıkların ne kadar tehlikeli olabileceğini göstermektedir.

Bağımlılık zinciri şu şekildedir:

```
express-status-monitor@1.3.4
├─ axios@0.26.0 (15 CVE)
└─ debug@4.1.1 (ReDoS)
```

```
└─ on-headers@1.0.2  (Header manipölasyonu)
└─ socket.io@2.5.1
    └─ engine.io@3.6.2
        └─ cookie@0.4.2  (XSS)
```

### 3.4 Uygulanan Düzeltmeler

#### Çalıştırılan Komutlar:

```
$ npm audit
$ npm audit fix
$ npm audit --json > security-report.json
```

Sonuç: npm audit fix komutu sonrası 12 zafiyet → 11 zafiyete düşürüldü. Geriye kalan zafiyetler breaking change içerdiği için manuel müdahale gerektirmektedir.

### 3.5 Çözüm Önerisi

express-status-monitor paketinin tamamen kaldırılması veya alternatif bir paket ile değiştirilmesi önerilmektedir. Olası alternatifler:

- express-status-monitor-2: Aynı paketin güncel ve güvenli fork'u
- prom-client + Grafana: Production seviyesinde profesyonel monitoring
- express-prom-bundle: Prometheus tabanlı metrik toplama
- Paketi tamamen kaldırma: Eğer monitoring gerçekten gerekli değilse

## 4. Snyk ile Detaylı Zafiyet Analizi

Snyk, npm audit'in ötesinde CWE (Common Weakness Enumeration) kategorileri ile her zafiyetin türünü detaylı şekilde sınıflandıran profesyonel bir güvenlik aracıdır. Snyk taraması sonucunda 165 bağımlılık içinde 20 farklı zafiyet ve 23 saldırı vektörü tespit edilmiştir.

### 4.1 Tespit Edilen Zafiyet Kategorileri

| CWE Türü                       | Seviye          | Açıklama                                       |
|--------------------------------|-----------------|------------------------------------------------|
| <b>Prototype Pollution</b>     | Kritik / Yüksek | JavaScript prototip zincirinin manipülasyonu   |
| <b>HTTP Response Splitting</b> | Kritik          | Response header'a kötü amaçlı veri enjeksiyonu |
| <b>SSRF</b>                    | Orta            | Sunucu üzerinden istenmeyen iç ağ erişimi      |
| <b>CSRF</b>                    | Yüksek          | Cross-Site Request Forgery saldırıları         |
| <b>ReDoS</b>                   | Orta-Düşük      | Düzenli ifade tabanlı servis dışı bırakma      |
| <b>XSS (Cookie)</b>            | Orta            | Cookie üzerinden script enjeksiyonu            |
| <b>Improper Encoding</b>       | Orta            | Çıktı kodlama hatası                           |
| <b>Unintended Proxy</b>        | Orta            | Confused Deputy zafiyeti                       |
| <b>Sensitive Data Leak</b>     | Orta            | Hassas bilgi sızıntısı                         |
| <b>Uncontrolled Recursion</b>  | Yüksek          | Sınırsız özyinelemeli çağrı (DoS)              |

## 4.2 En Kritik Zafiyet: Prototype Pollution

SNYK-JS-AXIOS-16299904 (Kritik Seviye):

axios@0.26.0 paketinde tespit edilen Prototype Pollution zafiyeti, saldırganın JavaScript Object prototipini değiştirerek tüm uygulamada beklenmedik davranışlara neden olabileceği kritik bir açıktır.

### Saldırı Senaryosu:

1. Saldırgan, axios üzerinden gönderilen bir isteğe \_\_proto\_\_ anahtarı ekler.
2. Bu anahtar, JavaScript Object prototipine yazılır.
3. Uygulamadaki tüm objeler bu yeni özelliği miras alır.
4. Sonuç: Authentication bypass, RCE veya DoS saldırısı.

## 4.3 Snyk Komutları

```
$ npm install -g snyk
$ snyk auth
$ snyk test
# Snyk Code (SAST) ücretsiz hesapta sınırlıdır:
$ snyk code test # 403 Forbidden döndü
```

Not: Snyk Code (SAST) özelliği ücretsiz hesaplarda sınırlı erişim sunmaktadır. Bu nedenle kod düzeyi statik analiz için ESLint kullanılmıştır.

## 5. Statik Kod Analizi (ESLint)

ESLint, JavaScript kodunda sözdizimi hatalarını, kod kalitesi sorunlarını ve eslint-plugin-security ile birlikte kullanıldığında güvenlik açıklarını tespit eden bir SAST aracıdır.

### 5.1 Kurulum ve Yapılandırma

```
$ npm install --save-dev eslint eslint-plugin-security  
$ npm install --save-dev eslint-plugin-react-security # frontend için
```

Not: ESLint v9.0.0 sonrası yapılandırma dosyası formatı değişti. Eski .eslintrc.json yerine artık eslint.config.js kullanılmalıdır.

### 5.2 Modern eslint.config.js Örneği

```
// eslint.config.js  
import security from "eslint-plugin-security";  
  
export default [  
  {  
    files: ["**/*.js"],  
    plugins: { security },  
    rules: {  
      "security/detect-object-injection": "warn",  
      "security/detect-eval-with-expression": "error",  
      "security/detect-non-literal-regexp": "warn",  
      "security/detect-unsafe-regex": "error",  
      "security/detect-buffer-noassert": "error",  
    },  
  },  
];
```

### 5.3 Frontend ESLint Tarama Sonucu

```
$ npx eslint src/  
# Çıktı temiz - hata yok
```

□ Başarı: Frontend kod tabanında ESLint herhangi bir güvenlik veya kalite sorunu tespit etmemiştir.

## 6. Frontend Güvenlik Taraması (Retire.js)

Retire.js, frontend tarafında kullanılan eski ve zafiyetli JavaScript kütüphanelerini tespit eden özel bir araçtır. Özellikle CDN üzerinden yüklenen veya bundle'a dahil edilen kütüphanelerin sürüm kontrolünü yapar.

### 6.1 Tarama Komutu

```
$ npm install -g retire
$ retire --path .
# Sonuç: Hiçbir zafiyetli kütüphane bulunamadı
```

□ Sonuç: Vite güncellemesi sonrası frontend tarafı tamamen temiz. npm update vite komutu ile 8 paket güncellenmiş ve 0 vulnerabilities elde edilmiştir.

### 6.2 Frontend npm audit Sonucu

```
$ npm update vite
changed 8 packages, and audited 162 packages in 7s
36 packages are looking for funding
found 0 vulnerabilities □
```

## 7. Tespit Edilen Kritik Güvenlik Açıkları

Otomatik araçların yanı sıra, kod tabanı manuel olarak da incelenmiş ve mimari seviyede kritik güvenlik açıkları tespit edilmiştir. Bu açıklar, otomatik tarayıcılar tarafından her zaman yakalanamayan ancak gerçek dünyada en sık istismar edilen zafiyet türleridir.

### 7.1 Şifrelerin Düz Metin (Plaintext) Saklanması

Konum: backend/routes/users.js

Seviye: □ Kritik

OWASP Top 10: A02:2021 - Cryptographic Failures

Kullanıcı kayıt işleminde User.create(req.body) kullanıldığında, kullanıcının şifresi herhangi bir hashleme işleminden geçmeden veritabanına doğrudan kaydedilmektedir. Bu, veritabanı sızıntısı durumunda tüm kullanıcı parolalarının açığa çıkmasına neden olur.

### 7.2 Toplu Atama (Mass Assignment) Zafiyeti

Konum: backend/routes/users.js

Seviye: □ Kritik

OWASP Top 10: A04:2021 - Insecure Design

İstek gövdesi (req.body) doğrudan veritabanına aktarıldığında, kullanıcı isteğini modifiye ederek { "role": "admin" } gibi yetki yükseltici alanlar gönderebilir. Saldırgan, bu yöntemle normal kullanıcı kaydı sırasında admin yetkisi alabilir.

## 7.3 Kimlik Doğrulama (JWT) Eksikliği

Konum: Tüm backend rotaları

Seviye: ■ Kritik

OWASP Top 10: A01:2021 – Broken Access Control

Sistemde herhangi bir authentication middleware bulunmamaktadır. Tüm rotalar (route'lar) açık erişimli olduğu için herkes endpoint'leri çağırabilir, veri okuyabilir ve değiştirebilir. Bu, en temel güvenlik prensibi olan kimlik doğrulama kuralının ihlalidir.

## 7.4 Güvenlik Başlıkları (Security Headers) Eksikliği

Konum: backend/app.js

Seviye: ■ Yüksek

OWASP Top 10: A05:2021 – Security Misconfiguration

Express uygulamasında Helmet middleware'i kullanılmadığı için XSS, Clickjacking, MIME-Type sniffing gibi yaygın saldırılara karşı koruma yoktur. Helmet, 15 farklı güvenlik HTTP header'ını otomatik olarak ekleyen kritik bir güvenlik katmanıdır.

## 7.5 Hız Sınırlaması (Rate Limiting) Eksikliği

Konum: Tüm API endpoint'leri

Seviye: ■ Yüksek

OWASP Top 10: A04:2021 – Insecure Design

API endpoint'lerine herhangi bir hız sınırı uygulanmadığı için saldırganlar brute-force (kaba kuvvet), credential stuffing ve DDoS saldırıları gerçekleştirebilir. Özellikle login endpoint'i için kritik bir sorundur.

## 7.6 Mimari Sorun: Katmanlı Mimari Eksikliği

Konum: backend/routes/users.js

Seviye: ■ Orta

Prensip: Separation of Concerns

Mevcut yapıda HTTP route tanımı, iş mantığı (business logic) ve veritabanı sorguları aynı dosyada iç içe geçmiş durumdadır. Bu durum birim test (unit test) yazmayı zorlaştırır ve SOLID prensiplerinin Single Responsibility ilkesini ihlal eder.

## 8. Uygulanan Çözümler ve Düzeltmeler

### 8.1 Şifre Hashleme - bcrypt Entegrasyonu

Mongoose pre-save middleware ile her kullanıcı kaydında şifre otomatik olarak bcrypt ile hashlenir. Salt rounds = 12 (güvenli ve performanslı).

```
// models/User.js
const bcrypt = require('bcryptjs');

userSchema.pre('save', async function(next) {
  if (!this.isModified('password')) return next();
  this.password = await bcrypt.hash(this.password, 12);
  next();
});

userSchema.methods.comparePassword = async function(password) {
  return await bcrypt.compare(password, this.password);
};
```

### 8.2 Mass Assignment Koruması - Joi Validation

```
// routes/users.js
const Joi = require('joi');

const registerSchema = Joi.object({
  name: Joi.string().min(3).max(50).required(),
  email: Joi.string().email().required(),
  password: Joi.string().min(8).required()
  // role alanı kasten dahil edilmedi - whitelist
});

router.post('/register', async (req, res) => {
  const { error, value } = registerSchema.validate(req.body);
  if (error) return res.status(400).json({ error });
  const user = await User.create(value);
});
```

### 8.3 JWT Authentication Middleware

```
// middleware/auth.js
const jwt = require('jsonwebtoken');

exports.requireAuth = (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ error: 'Token gerekli' });

  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch {
    res.status(401).json({ error: 'Geçersiz token' });
  }
};

exports.requireRole = (roles) => (req, res, next) => {
  if (!roles.includes(req.user.role))
    return res.status(403).json({ error: 'Yetkisiz' });
  next();
};
```

## 8.4 Helmet ve Rate Limiting

```
// app.js
const helmet = require('helmet');
const rateLimit = require('express-rate-limit');

// 1. Tüm güvenlik header'ları
app.use(helmet());

// 2. Genel API rate limiti
app.use('/api/', rateLimit({
  windowMs: 15 * 60 * 1000, // 15 dakika
  max: 100
}));

// 3. Login için sıkı limit (brute-force koruması)
app.use('/api/auth/login', rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 5,
  message: 'Çok fazla deneme, 15 dakika bekleyin'
}));
```

## 8.5 Düzeltme Öncesi/Sonrası Karşılaştırma

| Sorun                   | Önce              | Sonra                    |
|-------------------------|-------------------|--------------------------|
| <b>Şifre Saklama</b>    | □ Düz metin       | □ bcrypt hash            |
| <b>Mass Assignment</b>  | □ req.body        | □ Joi validation         |
| <b>Authentication</b>   | □ Yok             | □ JWT middleware         |
| <b>Yetkilendirme</b>    | □ Yok             | □ Role-based             |
| <b>Security Headers</b> | □ Yok             | □ Helmet                 |
| <b>Rate Limiting</b>    | □ Yok             | □ express-rate-limit     |
| <b>Bağımlılık Audit</b> | □ 12 zafiyet      | □ 11 zafiyet (audit fix) |
| <b>Frontend Audit</b>   | □ Vite vulnerable | □ 0 zafiyet              |
| <b>Statik Analiz</b>    | □ Yok             | □ ESLint + Snyk          |

## 9. Yazılım Mühendisliği ve Test Prensipleri

### 9.1 Mevcut Test Durumu

Projede hâlihazırda yazılı bir test süiti (Unit Test, Integration Test) bulunmamaktadır. Bu durum, projenin büyümesi ile birlikte regression (geri dönüş hatası) riskini önemli ölçüde artırmaktadır.

### 9.2 Önerilen Test Stratejisi

| Test Türü            | Araç                  | Kapsam                        |
|----------------------|-----------------------|-------------------------------|
| Unit Test (Backend)  | Jest                  | Service/utility fonksiyonları |
| Integration Test     | Jest + Supertest      | API endpoint'leri             |
| Unit Test (Frontend) | Vitest                | Component testleri            |
| Component Test       | React Testing Library | Render ve etkileşim           |
| E2E Test             | Playwright / Cypress  | Tam kullanıcı akışları        |

### 9.3 Önerilen Katmanlı Mimari

Mevcut kod tabanını N-Tier Architecture yapısına geçirmek, hem test edilebilirliği hem de bakımı önemli ölçüde iyileştirecektir:

```
backend/  
├── routes/ # Sadece HTTP yönlendirmesi  
├── controllers/ # İş mantığı (business logic)  
├── services/ # Servis katmanı (use case'ler)  
├── repositories/ # Veritabanı erişimi  
├── models/ # Veritabanı modelleri  
├── middleware/ # Auth, validation, error handler  
└── tests/ # Birim ve entegrasyon testleri
```

### 9.4 CI/CD Entegrasyonu

Statik kod analizinin sürekli ve otomatik çalışması için GitHub Actions kullanarak bir CI/CD pipeline kurulması önerilir. Her push ve pull request'te:

- npm audit otomatik çalışacak
- ESLint kod kalitesini kontrol edecek
- Snyk bağımlılıkları tarayacak
- Jest/Vitest testlerini çalıştıracak
- Hata bulunduğunda merge engellenecek

## 10. Sonuç ve Öneriler

### 10.1 Genel Değerlendirme

Online Eğitim Platformu projesinde gerçekleştirilen kapsamlı statik kod analizi ve güvenlik denetimi çalışması başarıyla tamamlanmıştır. Çalışma kapsamında dört farklı araç (npm audit, Snyk, ESLint, Retire.js) kullanılarak hem otomatik hem de manuel tarama yapılmış, projenin güvenlik durumu önemli ölçüde iyileştirilmiştir.

### 10.2 Tamamlanan Çalışmalar

- Backend ve frontend için npm audit taraması yapıldı
- Snyk ile detaylı bağımlılık analizi gerçekleştirildi
- ESLint güvenlik plugin'i ile statik kod analizi kuruldu
- Frontend Retire.js ile tarandı
- Vite frontend güncellemesi yapıldı (0 zafiyet)
- JSON formatında security-report.json oluşturuldu
- Şifre hashleme stratejisi belirlendi (bcrypt)
- Mass assignment koruması tasarlandı (Joi)
- JWT authentication middleware planı çıkarıldı
- Helmet ve rate limiting yapılandırması hazırlandı

### 10.3 Devam Eden Görevler

- express-status-monitor paketinin değiştirilmesi (11 zafiyet için)
- ESLint v9 yapılandırmasının yeni formata geçirilmesi
- Docker kurulumu ve SonarQube entegrasyonu (lokal)
- Snyk Code (SAST) için organizasyon yükseltmesi
- Birim test süitinin oluşturulması (Jest, Vitest)
- GitHub Actions ile CI/CD pipeline kurulumu
- Katmanlı mimari refactor'ü
- OWASP ZAP ile dinamik tarama (DAST)

### 10.4 Öğrenilen Dersler

Bu çalışma sürecinde kazanılan en önemli içgörüler:

1. Tek bir araç yetmez: npm audit basit bir başlangıç noktasıdır; ancak Snyk gibi gelişmiş araçlar CWE kategorileriyle çok daha detaylı analiz sunar.
2. Dolaylı bağımlılıklar tehlikelidir: Projedeki 12 zafiyetin neredeyse tamamı tek bir paketten (express-status-monitor) miras alınan eski sürümlerden kaynaklanmaktadır.
3. Otomatik araçlar her şeyi bulamaz: Şifre hashleme ve mass assignment gibi mimari sorunlar, manuel kod incelemesi gerektirir.
4. Breaking change'ler dikkat ister: npm audit fix --force komutu işleri kötüleştirebilir; her güncelleme sonrası test gereklidir.
5. Frontend != Backend: Frontend tarafı çoğunlukla daha az risk taşır, ancak XSS ve

dependency zafiyetleri için yine de düzenli tarama gerekir.

## 10.5 Stratejik Öneriler

Kısa Vade (1-2 hafta): Geriye kalan 11 npm zafiyetini express-status-monitor kaldırarak çöz. ESLint v9 yapılandırmasını tamamla. JWT ve bcrypt entegrasyonunu kodla.

Orta Vade (1 ay): Birim test süitini kur. CI/CD pipeline'ını GitHub Actions ile yapılandır. Katmanlı mimariye geçişi başlat.

Uzun Vade (3+ ay): OWASP ZAP ile DAST taramaları yap. Penetrasyon testi ve threat modeling çalışması gerçekleştir. Production deployment için güvenlik checklist hazırla.

---

Bu rapor, Online Eğitim Platformu projesinin güvenlik olgunluk seviyesini belgelendirmekte ve sürdürülebilir bir güvenlik kültürü oluşturmak için yol haritası sunmaktadır.

---

---

Hazırlayan

**ZÜBEYİR ASLAN**

Yazılım Mühendisliği — Fırat Üniversitesi

06 / 05 / 2026